FORMFIT

AI-Powered Exercise Form Trainer

Complete Beginner's Guide & Project Documentation

Built with Python | Flask | OpenCV | MediaPipe | React

PART 1: What Is This Project?

1. The Big Picture — What Does FormFit Do?

FormFit is an AI-powered web application that watches you exercise through your webcam and tells you in real time whether your form is correct. Think of it like having a personal trainer watching you from your screen — it counts your reps, checks your posture, and warns you when you are doing something wrong that could cause injury.

For example, when you do a push-up:

- It detects where your arms, shoulders, elbows, and hips are
- It measures the angle of your elbows
- It counts the rep only when you do a full push-up correctly
- It warns you if your elbows are flaring out (a common mistake that causes shoulder injury)

Why does this matter?

Most people at the gym do exercises with wrong form — not because they are lazy, but because nobody is watching them and correcting them. A personal trainer costs hundreds of rupees per hour. FormFit does the same job for free, using just a webcam and AI.

1.1 What Exercises Does It Support?

Exercise	Muscles Targeted	What FormFit Checks
Push-Ups	Chest, Triceps, Shoulders	Elbow angle, back straightness, elbow flare
Squats	Quads, Glutes, Hamstrings	Knee angle, depth, knee caving
Bicep Curls	Biceps, Forearms	Full range of motion, elbow drift
Shoulder Press	Deltoids, Triceps, Traps	Full arm extension, controlled lowering

PART 2: System Architecture

2. How the System is Built — The Architecture

Architecture means the overall structure of how the project is organized — which parts exist, what each part does, and how they talk to each other. Think of it like the blueprint of a building.

2.1 The Three-Layer Architecture

Simple Analogy

Imagine a restaurant. The customer sits at the table (Frontend / React). They place an order with the waiter (API). The chef in the kitchen prepares the food (Backend / Flask + MediaPipe). The customer never goes into the kitchen, and the chef never comes to the table — they communicate through the waiter.

Layer	Technology	What It Does
Frontend (UI)	React (JavaScript)	The webpage you see — home page, exercise cards, live video panel, stats
Backend (Server)	Flask (Python)	Receives requests from the browser, processes camera, sends back data
AI Engine	MediaPipe + OpenCV	Detects body landmarks, calculates angles, decides if form is correct

2.2 How the Parts Communicate

The Frontend (React) and Backend (Flask) run as two separate programs but talk to each other using something called an API (Application Programming Interface). Think of API as a set of rules for how two programs can talk to each other.

API Endpoint (URL)	Method	What It Does
/api/video/pushups	GET	Streams live webcam video with pose overlay drawn on it
/api/data/pushups	GET	Returns rep count, stage, feedback message as JSON data
/api/reset/pushups	POST	Resets the rep counter back to zero
/api/health	GET	Simple check to see if the server is running

2.3 Data Flow — What Happens When You Open An Exercise

Here is the exact sequence of events, step by step:

1. You click 'Push-Ups' on the home page in your browser
2. React navigates to /trainer/pushups and renders the trainer page
3. The page sends a request to http://localhost:5000/api/video/pushups
4. Flask starts your webcam using OpenCV (cv2.VideoCapture(0))
5. MediaPipe analyzes each frame and detects 33 body landmarks
6. The angle calculator measures elbow, back, and flare angles
7. The state machine decides your current stage and whether form is correct
8. OpenCV draws the skeleton, rep count, and feedback text on the frame
9. Flask compresses the frame as JPEG and streams it to your browser
10. React also polls /api/data/pushups every 200ms to update the stats panel

PART 3: The Backend — app.py Explained

3. The Backend — app.py (Python)

The backend is the brain of the project. It is written in Python and uses several libraries. Let us understand what a library is first.

What is a Library?

A library is a collection of pre-written code that someone else wrote, so you do not have to write everything from scratch. For example, instead of writing your own code to open a webcam, you just use the OpenCV library which already knows how to do that. pip install is the command to download and install these libraries.

3.1 Libraries Used and Why

Library	What It Is	Why We Use It
Flask	A lightweight web server framework for Python	Creates the API endpoints that React talks to. Receives HTTP requests and sends responses.
flask_cors	Cross-Origin Resource Sharing handler	Allows React (running on port 3000) to talk to Flask (running on port 5000). Without this, the browser blocks the connection for security reasons.
OpenCV (cv2)	Computer Vision library by Intel	Opens the webcam, reads video frames, draws rectangles and text on the video, and encodes frames as JPEG images.
MediaPipe	AI pose estimation library by Google	Detects 33 body landmarks (joints) in real-time from video frames. This is the core AI that makes the project work.
NumPy	Numerical computing library	Used for mathematical calculations — specifically the trigonometry needed to calculate angles between joints.
threading	Built into Python	Allows multiple things to happen at the same time (e.g., processing video while also serving API requests).

3.2 The Flask App Setup

```
app = Flask(__name__) CORS(app)
```

Flask(__name__) creates the web server application. __name__ is a special Python variable that tells Flask the name of the current file so it knows where to find things. CORS(app) enables cross-origin requests — this is necessary because your browser runs React on port 3000 and Flask on port 5000, and browsers block communication between different ports by default for security.

3.3 The Global State Dictionary

Every exercise has a 'state' — a collection of variables that track what is currently happening. This is stored in a Python dictionary (a key-value store, like a real-world dictionary where each word has a definition):

```
exercise_states = {     "pushups": {         "count": 0,              # Number of
reps completed         "stage": "GET_READY",    # Current stage in state machine
"feedback": "...",       # Message shown on screen         "feedback_color":
"yellow",  # Color of the message         "angle_debug": {}        # Current angle
values     },     "squats": { ... },     "bicep_curls": { ... },
"shoulder_press": { ... } }
```

Why Global State?

We use a global dictionary so that the angle analyzer function and the API endpoint function can both access the same data. The analyzer updates the count and feedback, and the API endpoint reads those values and sends them to React. They share the same dictionary in memory.

3.4 MediaPipe Setup

```
mp_pose = mp.solutions.pose mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles
```

mp.solutions.pose is the pose estimation model from MediaPipe. It can detect 33 body landmarks from a single camera image. mp_drawing is a helper that draws the skeleton (the green dots and lines you see on the video). These are initialized once at the top of the file so they are ready to use throughout the program.

4. The Angle Calculation — The Heart of the AI

This is the most important function in the entire project. Everything else depends on this.

4.1 How Angles Are Calculated

```
def calculate_angle(a, b, c):     a, b, c = np.array(a), np.array(b), np.array(c)
radians = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1], a[0]-b[0])
angle = np.abs(np.degrees(radians))     return 360 - angle if angle > 180 else
angle
```

Simple Explanation

Imagine three dots on a piece of paper: A (shoulder), B (elbow), C (wrist). The calculate_angle function finds the angle at point B — the elbow. It uses trigonometry (arctan2 function) to measure the opening angle. For example, when your arm is straight, the angle is about 170-180 degrees. When fully bent, it is about 30-50 degrees. This number tells us whether your arm is up or down.

The function takes three points (each is a list of [x, y] coordinates on the screen):

- a = the first joint (e.g., shoulder)
- b = the middle joint where we measure the angle (e.g., elbow)
- c = the last joint (e.g., wrist)
- np.array() converts the points into NumPy arrays for math operations
- np.arctan2() calculates the angle in radians using the x,y coordinates
- np.degrees() converts radians to degrees (0 to 360)
- The final line ensures the angle is always the smaller of the two possible angles

4.2 Getting Landmark Coordinates

```
def get_landmark(landmarks, idx, w, h):     lm = landmarks[idx]     return [lm.x *
w, lm.y * h]
```

MediaPipe returns landmark positions as values between 0 and 1 (called normalized coordinates). For example, x=0.5 means the middle of the screen horizontally. We multiply by the actual width (w) and height (h) of the video frame to get the real pixel position. The idx parameter is the landmark ID number — MediaPipe assigns a specific number to each body part.

Landmark ID	Body Part	Used For
LEFT_SHOULDER (11)	Left shoulder joint	Elbow angle calculation, back angle
LEFT_ELBOW (13)	Left elbow joint	Primary angle measurement point
LEFT_WRIST (15)	Left wrist	Completing the elbow angle triangle

LEFT_HIP (23)	Left hip joint	Back straightness, squat depth
LEFT_KNEE (25)	Left knee joint	Squat knee angle measurement
LEFT_ANKLE (27)	Left ankle	Completing the knee angle triangle

5. The State Machine — How Reps Are Counted

A state machine is one of the most important concepts in this project. It is a system that can be in one of several 'states' and moves between states based on certain conditions.

> **Real-World Analogy**
> Think of a traffic light. It has 3 states: Red, Yellow, Green. It moves from Green to Yellow when a timer expires, from Yellow to Red when another timer expires, and from Red to Green after a third timer. It follows rules — it never goes from Red directly to Yellow. Our exercise tracker works the same way.

5.1 Push-Up State Machine

The push-up tracker has 3 states:

State	Meaning	Condition to Move to Next State
GET_READY	User is not in position yet	Back angle > 160 degrees AND elbow angle > 150 degrees (plank position detected)
READY	User is in plank, arms extended	Elbow angle drops below 90 degrees (going down)
DOWN	User is at the bottom of push-up	Elbow angle rises above 150 degrees (pushing back up) — REP IS COUNTED HERE

```
if state['stage'] == 'GET_READY':     if back_angle > 160 and avg_elbow > 150:
state['stage'] = 'READY'         state['feedback'] = 'In position! Go down' elif
state['stage'] == 'READY':     if avg_elbow < 90:         state['stage'] = 'DOWN'
state['feedback'] = 'Push up!' elif state['stage'] == 'DOWN':     if avg_elbow >
150:         state['stage'] = 'READY'         state['count'] += 1   # <-- REP
COUNTED!
```

Why do we need all three states? Because without a state machine, the program would count a rep every single frame where the arm is at a certain angle — that could be 30 times per second! The state machine ensures a rep is only counted once per complete up-down-up cycle.

5.2 Form Checking — Elbow Flare Detection

```
left_flare = calculate_angle(lh, ls, le)  # hip-shoulder-elbow angle form_ok =
left_flare < 55
```

The flare angle measures how far your elbows are pointing outward. The angle is measured between your hip, shoulder, and elbow. If this angle is greater than 55 degrees, your elbows are flaring out — a common mistake that puts stress on your shoulder joints. When form_ok is False, the feedback changes to a warning message.

5.3 Squat State Machine

State	Meaning	Condition to Move
STAND	Initial state, standing up	Knee angle > 160 degrees
UP	Standing, ready to squat	Knee angle drops below 100 degrees (squat detected)
DOWN	At bottom of squat	Knee angle rises above 160 degrees — REP COUNTED

6. The Video Streaming System

This is how the live webcam feed gets from your camera to your browser. It uses a technique called MJPEG streaming (Motion JPEG).

6.1 How MJPEG Streaming Works

Simple Analogy

Imagine a flipbook animation. Instead of playing a video file, we send one image at a time, very fast. Each image is a JPEG photo of your webcam. When you receive 20-30 of them per second, your brain sees it as smooth video. MJPEG (Motion JPEG) is exactly this — a rapid sequence of JPEG images sent over the internet.

```
def generate_frames(exercise_id):     with mp_pose.Pose(...) as pose:         cap
= cv2.VideoCapture(0)  # Open webcam (0 = first camera)         while True:
ret, frame = cap.read()    # Read one frame             frame = cv2.flip(frame, 1)
# Mirror flip (like a selfie)             # ... process frame ...             _,
buffer = cv2.imencode('.jpg', frame)  # Convert to JPEG             yield (b'--
frame\r\n'                   # Send to browser                     b'Content-Type:
image/jpeg\r\n\r\n'                     + buffer.tobytes() + b'\r\n')
```

Line by line explanation:

- cv2.VideoCapture(0) — Opens the default webcam. The 0 means 'first camera'. If you have multiple cameras, you might use 1 or 2.
- cap.read() — Grabs one frame from the webcam. Returns ret (True if successful) and frame (the actual image as a NumPy array).
- cv2.flip(frame, 1) — Flips the image horizontally so it works like a mirror (selfie mode). Without this, left and right are reversed.
- cv2.imencode('.jpg', frame) — Compresses the frame as a JPEG file in memory. Quality is set to 80% to save bandwidth.
- yield — This is a Python generator. Instead of returning one result and stopping, yield sends one frame and then continues the loop to produce the next frame.

6.2 Drawing On The Video

OpenCV draws the UI elements directly on each video frame before sending it to the browser:

```
# Draw skeleton mp_drawing.draw_landmarks(frame, results.pose_landmarks, ...)  #
Draw dark top bar cv2.rectangle(overlay, (0, 0), (w, 90), (15, 15, 25), -1)  #
Draw rep counter cv2.putText(overlay, str(state['count']), (55, 75),
cv2.FONT_HERSHEY_SIMPLEX, 1.8, (0, 255, 150), 3)  # Draw feedback bar at bottom
cv2.rectangle(overlay, (0, h-70), (w, h), (15, 15, 25), -1) cv2.putText(overlay,
state['feedback'], (20, h-25), ...)
```

cv2.rectangle(image, top_left_point, bottom_right_point, color, thickness) — Draws a filled rectangle. The -1 thickness means fill the rectangle completely. Colors are in BGR format (Blue, Green, Red) not RGB — this is an OpenCV quirk.

cv2.putText(image, text, position, font, size, color, thickness) — Draws text on the frame. The position is the bottom-left corner of where the text starts.

cv2.addWeighted(overlay, 0.9, frame, 0.1, 0, frame) — Blends the overlay with the original frame for a semi-transparent effect. 0.9 means 90% overlay and 10% original.

PART 4: The Frontend — React Explained

7. The Frontend — React (JavaScript)

The frontend is everything you see in the browser. It is built with React, a JavaScript library for building user interfaces. Let us start with the basics.

7.1 What is React?

What is React?

React is a library that lets you build web pages by creating reusable 'components' — small, self-contained pieces of UI. Think of components like LEGO bricks. A Button component is one brick, a Card component is another brick, and you assemble them to build the full page. React automatically updates the page when data changes, without you having to manually refresh anything.

7.2 Project File Structure

```
formfit_clean/   app.py              <-- Backend (Python)   requirements.txt    <-
- List of Python libraries to install   package.json        <-- List of JavaScript
libraries to install   public/     index.html        <-- The HTML shell that React
loads into   src/     index.js          <-- Entry point: attaches React to
index.html     index.css         <-- Global styles (dark theme colors)     App.js
<-- Router: decides which page to show     pages/       HomePage.js     <-- The
home page with exercise cards       HomePage.css    <-- Styling for the home page
TrainerPage.js  <-- The live trainer view       TrainerPage.css <-- Styling for
the trainer page
```

7.3 How React Starts — index.js

```
import React from 'react'; import ReactDOM from 'react-dom/client'; import
'./index.css'; import App from './App';  const root =
ReactDOM.createRoot(document.getElementById('root')); root.render(<App />);
```

This is the entry point — the first JavaScript file that runs. ReactDOM.createRoot finds the element in index.html and hands control to React. Then React renders (draws) the App component inside that div. The whole webpage lives inside that one div.

7.4 The Router — App.js

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';  export default
function App() {   return (     <BrowserRouter>       <Routes>         <Route
path='/' element={<HomePage />} />         <Route path='/trainer/:exerciseId'
element={<TrainerPage />} />       </Routes>     </BrowserRouter>   ); }
```

React Router handles navigation without reloading the page. When you are at '/' (the root URL), it shows HomePage. When you navigate to '/trainer/pushups', it shows TrainerPage and passes 'pushups' as the exerciseId parameter. The :exerciseId part is a dynamic variable — it changes depending on which exercise you clicked.

8. The Home Page — HomePage.js

The home page shows the landing screen with exercise cards. Let us walk through its main parts.

8.1 Exercise Data Array

```
const EXERCISES = [   {     id: 'pushups',     name: 'Push-Ups',     muscles:
'Chest, Triceps, Shoulders',     difficulty: 'Beginner',     color: '#00ff87',
// ...   },   // ... 3 more exercises ];
```

Instead of writing four separate card components, we store all exercise data in one array and loop through it. This makes it easy to add new exercises later — just add a new object to the array, and a card automatically appears on the page.

8.2 The Animated Background — Canvas

```
const canvasRef = useRef(null);  useEffect(() => {   const canvas =
canvasRef.current;   const ctx = canvas.getContext('2d');   // Create 60 random
floating particles   const particles = Array.from({ length: 60 }, () => ({     x:
Math.random() * canvas.width,     y: Math.random() * canvas.height,     vx:
(Math.random() - 0.5) * 0.3,  // velocity     vy: (Math.random() - 0.5) * 0.3,
}));   // Animation loop   const draw = () => {     // Move each particle and
redraw     requestAnimationFrame(draw);   };   draw(); }, []);
```

useRef creates a reference to the canvas HTML element. useEffect runs code after the component appears on screen — perfect for starting animations. requestAnimationFrame is the browser's built-in way to run code 60 times per second for smooth animations. The particles are small green dots that float around in the background, giving the page a tech/AI aesthetic.

8.3 Card Click Navigation

```
const navigate = useNavigate();  // Inside the card: onClick={() =>
navigate('/trainer/' + ex.id)}
```

useNavigate is a React Router hook that gives you a function to change the page. When you click a card, navigate('/trainer/pushups') changes the URL to /trainer/pushups, which triggers the Router in App.js to show the TrainerPage component with exerciseId = 'pushups'.

9. The Trainer Page — TrainerPage.js

This is the most complex part of the frontend. It shows the live camera feed, polls for real-time data, and displays the stats panel.

9.1 Reading the URL Parameter

```
const { exerciseId } = useParams(); // If URL is /trainer/pushups, exerciseId =
'pushups' // If URL is /trainer/squats, exerciseId = 'squats'
```

useParams is a React Router hook that reads the dynamic part of the URL. This single TrainerPage component handles all 4 exercises — it just changes its behaviour based on what exerciseId is.

9.2 Polling for Live Data

```
useEffect(() => {   // Run this every 200 milliseconds   pollRef.current =
setInterval(async () => {     const res = await fetch('/api/data/' + exerciseId);
if (res.ok) {       const json = await res.json();       setData(json);  // Update
the stats panel     }   }, 200);    return () => clearInterval(pollRef.current);
// Cleanup }, [exerciseId]);
```

setInterval runs a function repeatedly every X milliseconds. Every 200ms (5 times per second), we call fetch() to request the latest data from Flask. The response is JSON (JavaScript Object Notation) — a text format for sending structured data. setData updates the React state, which automatically re-renders the stats panel with new numbers.

> **Why 200ms and not 1000ms?**
> 200ms (5 times per second) gives a responsive feel — the rep counter updates quickly after each rep. We do not poll faster because the video stream already handles visual feedback. We also do not poll at 60fps because that would be too many requests and could slow down the server.

9.3 Displaying the Video Stream

```
<img   src={"/api/video/" + exerciseId}   alt="Live camera feed"
className="camera-feed" />
```

The video stream is displayed using a simple HTML img tag, not a video tag. This works because the browser supports MJPEG streams natively as image sources. When the browser requests this URL, Flask keeps the connection open and keeps sending JPEG frames, one after another. The browser displays each new frame as it arrives, creating the illusion of live video.

9.4 The Session Timer

```
const [elapsed, setElapsed] = useState(0); const [sessionStart] =
useState(Date.now());  timerRef.current = setInterval(() => {
setElapsed(Math.floor((Date.now() - sessionStart) / 1000)); }, 1000);  // Format:
00:45 format const formatTime = (s) =>   String(Math.floor(s/60)).padStart(2,'0')
+ ':' +   String(s % 60).padStart(2,'0');
```

Date.now() returns the current time in milliseconds since January 1, 1970 (called Unix timestamp). By subtracting sessionStart from Date.now() every second, we get the elapsed time. padStart(2, '0') makes sure single digits show as '01' instead of '1' — so 1 minute 5 seconds shows as '01:05' not '1:5'.

PART 5: Why We Made These Choices

10. Key Design Decisions — Why We Did Things This Way

10.1 Why Python for the Backend?

MediaPipe and OpenCV are primarily Python libraries. They have the best support, documentation, and community in Python. While they have JavaScript versions, the Python versions are more mature, more accurate, and better documented. Since the AI processing is the core feature, we built the backend in Python to get the best performance.

10.2 Why React for the Frontend?

React is the most popular frontend framework in the world. It makes building interactive UIs much easier than plain HTML/JavaScript because it automatically updates the page when data changes. For example, when the rep count changes, React automatically re-renders just the counter number without refreshing the whole page.

10.3 Why Flask Instead of Django?

Flask is lightweight and minimalist — perfect for a simple API server. Django (the other major Python web framework) is much more complex and is designed for full web applications with databases, user authentication, etc. Since we only need a few API endpoints and video streaming, Flask is the right tool.

10.4 Why MJPEG Instead of WebRTC?

WebRTC (Web Real-Time Communication) is the 'proper' way to stream video in web browsers and has very low latency. However, it is extremely complex to set up and requires a STUN/TURN server. MJPEG is much simpler — just sending JPEG images over HTTP — and works perfectly for this use case. The slight latency is acceptable since we are doing form checking, not gaming.

10.5 Why Poll Every 200ms Instead of WebSockets?

WebSockets allow real-time two-way communication and would be more efficient for live data. However, they require more complex setup. Since the video stream already gives visual feedback, the 200ms polling for the stats panel works well in practice and is much simpler to implement and understand.

10.6 Why Two Separate Processes (Backend + Frontend)?

Running them separately gives us clean separation of concerns — the backend only worries about camera and AI processing, the frontend only worries about UI. This also means in the future, you could replace the React frontend with a mobile app, or deploy the backend to a server and the frontend to a CDN, without changing any code.

PART 6: Common Questions & Answers

11. Questions You Might Be Asked

Q: What is MediaPipe and how does it work?

MediaPipe is an open-source AI framework developed by Google. It uses a machine learning model that was trained on millions of images of human bodies. When given a camera frame, it uses this trained model to predict where 33 body landmarks are — shoulders, elbows, wrists, hips, knees, ankles, etc. It does this entirely on your device (no data is sent to Google) and is fast enough to run in real-time at 30+ frames per second.

Q: How accurate is the form detection?

The form detection is based on geometric angle thresholds that are well-established in exercise science. For example, it is widely accepted that elbow flare above 45-60 degrees in push-ups is problematic. The system is reasonably accurate for standard body types and good lighting, but it can struggle with partially visible bodies, extreme camera angles, or very fast movements.

Q: What is the proxy setting in package.json?

```
"proxy": "http://localhost:5000"
```

Without the proxy, React would try to fetch /api/video/pushups from localhost:3000 (its own address), which would fail because Flask is on port 5000. The proxy setting tells React's development server to forward any request that starts with /api to localhost:5000 automatically. This is a development-only feature — in production, you would configure a web server like Nginx to do the same thing.

Q: Why do we need npm install and pip install?

npm install reads the package.json file and downloads all the JavaScript libraries the React app needs. pip install reads requirements.txt and downloads all the Python libraries the backend needs. These libraries are not included in the project files themselves because they would make the project folder gigabytes in size. Instead, we just list the names and versions, and the package managers download them fresh.

Q: What is a virtual environment (venv)?

A virtual environment is an isolated Python installation for your project. Without it, all Python projects on your computer share the same libraries. This causes problems when different projects need different versions of the same library. With venv, each project has its own private

copy of Python and its libraries. 'venv\Scripts\activate' switches your terminal to use this project's Python instead of the system Python.

Q: How would you add a new exercise?

To add, say, 'Lunges', you would need to do 4 things:

11. In app.py: Add 'lunges' to exercise_states dictionary with initial values
12. In app.py: Write an analyze_lunges() function with the angle logic and state machine
13. In app.py: Add 'lunges': analyze_lunges to the ANALYZERS dictionary
14. In HomePage.js: Add a new object to the EXERCISES array with the exercise details
15. In TrainerPage.js: Add the 'lunges' entry to EXERCISE_INFO with tips and tracked angles

That is all! The rest of the system (routing, streaming, polling) handles it automatically.

Q: Why do we use threading?

Flask by default handles one request at a time. But our app has multiple requests happening simultaneously — the video stream is a long-running connection that never closes, while the /api/data endpoint needs to respond every 200ms. threading=True in app.run() allows Flask to handle multiple requests in parallel threads, so one long connection does not block others.

Q: What is JSON?

JSON (JavaScript Object Notation) is a text format for sending structured data between programs. It looks like a Python dictionary. For example, the /api/data endpoint returns: {"count": 5, "stage": "READY", "feedback": "Keep going!"}. React receives this text, parses it back into a JavaScript object, and uses the values to update the stats panel.

12. Summary — The Complete Picture

FormFit is a full-stack AI web application that demonstrates how multiple technologies work together to solve a real problem. Here is the complete flow one more time:

16. You open the browser at localhost:3000 — React serves the home page
17. You click Push-Ups — React Router navigates to /trainer/pushups
18. The TrainerPage component renders and makes two connections to Flask on port 5000
19. Connection 1: An img tag requests /api/video/pushups — Flask opens your webcam
20. MediaPipe processes each frame, calculating joint angles 30 times per second
21. The state machine uses those angles to detect stages and count reps
22. OpenCV draws the skeleton, rep count, stage, and feedback on each frame
23. Each frame is encoded as JPEG and streamed to your browser via MJPEG
24. Connection 2: setInterval calls /api/data/pushups every 200ms
25. Flask returns the current count, stage, and feedback as JSON
26. React updates the stats panel with the new data — no page refresh needed

The Key Takeaway

The power of this project comes from combining the right tools for the right jobs. Python/MediaPipe for AI processing. Flask for serving data over HTTP. React for building a reactive user interface. OpenCV for video processing. Each tool does what it is best at, and together they create something that would be very difficult to build with any single technology alone.

Concept	Technology Used	Why It Matters
Pose Estimation	Google MediaPipe	The entire project depends on accurate landmark detection
Angle Math	NumPy arctan2	Converts landmark positions into meaningful exercise metrics
State Machine	Python if/elif logic	Prevents false rep counts and enforces correct movement sequence
Video Streaming	MJPEG over HTTP	Gets processed video from Python to the browser simply
API Communication	Flask REST API	Allows React and Python to share data over HTTP
Reactive UI	React useState/useEffect	Automatically updates the page when data changes
Real-time Polling	setInterval + fetch	Keeps the stats panel in sync with the backend state